

GAUTENG CITY COLLEGE

ADVANCED PROGRAMMING II

DATA VALIDATION SYSTEM JAVA SWING

Student Name: LOYISO SONTI

Student Number: 0107145922087

Lecturer: Mr Owami

Date: 22 September 2026

Table of contents

1. Introduction	
2. System Design	
3. Fields	
4. Validation Checks	
6 5. Testing	
6. Code Steps.....	
7. GitHub Repository	
8. Conclusion	
9. References	

Introduction

For this project, I created a **Data Validation System using Java** . The main purpose of the system is to collect information from a user and check whether the information entered is correct.

The system allows the user to enter their name and surname, student number, date of birth, gender, contact number and email address.

Validation is important because it helps prevent users from entering incorrect information. For example, the system can check that a required field is not empty, that numbers are entered where numbers are expected, and that an email address is in the correct format.

The system also gives the user a clear message when something is wrong so that they know what needs to be fixed.

System Design

Before creating the application, I planned the layout so that it would be simple and easy to understand.

I arranged the fields in a logical order and used clear labels so that the user knows what information needs to be entered. I also added **Validate & save, Clear and Exit** buttons for the main functions of the application.

I used suitable colours, fonts and spacing instead of keeping the basic default Java design. My main goal was to make the application look neat while still keeping it simple.

DATA VALIDATION SYSTEM

._***._ *THANK YOU FOR TRUSTING US* .***._

Name:

Surname:

Student Number:

Date of birth:

Gender: ☐ Male ☐ Female ☐ Other

Contact Number:

EmailAddress:

VALIDATE & SAVE

CLEAR

EXIT

The screenshot shows the main form and the different fields that the user needs to complete.

Fields

ID / Student Number

This field is used to enter the user's ID or student number. The system checks that the number is entered correctly and that it has the required length.

Date of Birth

This field is used to enter the user's date of birth. The date must follow the format:

YYYY-MM-DD

Gender

The user selects their gender using radio buttons. This makes it easier for the user to choose an option instead of typing it manually.

Contact Number

This field is used to enter the user's contact number. The system checks that the number contains the correct type of characters.

Email Address

This field is used to enter the user's email address. The system checks that the email follows a valid format.

Validation Checks

I used different validation checks because not all fields require the same type of checking.

Presence Check

The presence check makes sure that the user does not leave a required field empty.

For example, if I click **Save** without entering a name, the system tells me:

"Name is required."

This lets the user know exactly what needs to be completed.

Type Check

The type check makes sure that the user enters the correct type of information.

For example, a contact number should contain numbers and not letters.

If I enter:

082A661234

the system will show an error telling me that the contact number must contain numbers only.

Length Check

The length check makes sure that the information is not too short or too long.

For example, the student number must contain the required number of characters. If the user enters too few characters, the system will display an error.

Format Check

The format check makes sure that information follows the expected pattern.

For example, the date of birth must be entered as:

YYYY-MM-DD

An email address should also have a valid format, such as:

student@outlook.com

Range Check

The range check is used for values such as age. It makes sure that the number entered is within a reasonable range.

This prevents the user from entering values that do not make sense.

Testing

After creating the system, I tested the different fields to make sure that the validation worked correctly.

I tested both **incorrect information** and **correct information**.

Name Testing

First, I left the name field empty and clicked the Save button.

Empty name field

The system displayed an error message telling me that the name is required.

I then entered a valid name and tested it again.

Valid name entered

The system accepted the name.

Student Number Testing

I tested the student number by entering incorrect information first.

Invalid student number

The system detected the problem and displayed an error message.

I then entered a valid student number.

Valid student number

The system accepted the information.

Date of Birth Testing

I first entered the date in the wrong format.

Incorrect date format

The system informed me that the date needed to follow the correct format.

I then entered a valid date.

Correct date of birth

The system accepted the date. As the system also tells the user that the date can not be in future tense.

Gender Testing

I tested what happens when no gender is selected.

Gender not selected

The system displayed a message asking me to select a gender.

I then selected an option and tested the form again.

Gender selected

The system accepted the selection.

Contact Number Testing

I entered letters into the contact number field to see if the system would detect the error.

Invalid contact number

The system displayed an error message explaining that the contact number must contain numbers.

I then entered a valid contact number.

Valid contact number

The system accepted the number.

Email Testing

I tested the email field using an incorrect email format.

For example:

loyds.com

Invalid email address

The system displayed an error message.

I then entered a valid email address.

student@outlook.com@

Valid email address

The system accepted the email.

Final Testing

After testing the fields separately, I entered valid information into the entire form.

Completed form with valid information

All the fields passed the validation checks and the system displayed a successful message.

This showed that the application was working correctly when all the required information was entered.

Code Steps

I divided the program into different parts so that the code would be easier to understand and manage.

Main Application

The main class is responsible for starting the application. It opens the Data Validation System window when the program is run.

Validator Class

The Validator class contains the different checks used by the application.

For example, it checks whether fields are empty, whether numbers contain the correct characters, whether dates are valid and whether email addresses follow the correct format.

Save Button

The Save button starts the validation process. When the user clicks it, the system checks the information entered into the different fields.

If something is wrong, an error message is shown. If everything is correct, the system confirms that the information is valid.

Clear Button

The Clear button removes the information currently entered in the form. This allows the user to start again without closing the application.

Exit Button

The Exit button closes the application when the user is finished.

GitHub Repository

I uploaded my project to a public GitHub repository so that the project can be viewed and downloaded.

To run the project, a marker can clone the repository and open it using NetBeans. The project can then be built and run from the IDE.

The README file also explains what the project does and provides basic instructions on how to run it.

Creating this project helped me improve my understanding of **Java and data validation**.

One of the main challenges I experienced was making sure that each field handled incorrect information properly. I had to test different types of input to make sure that the system could identify missing, incorrect or invalid information.

I also learned that the design of an application is important. The program should not only work correctly, but should also be easy for the user to understand.

Overall, this project gave me practical experience in developing a Java application and using validation to make sure that user input is correct.

References

Apache NetBeans.